DATABASES

Sharding in Practice Visual Guide

Shard keys, resharding, Vitess, Citus -- scaling out without losing your mind.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

When to Shard

Sharding is the last resort -- recognize the real signals.

Write volume

single primary cannot keep up, even with bigger hardware

Data size

dataset exceeds what one node holds / backs up sanely

Connection limits

thousands of app connections overwhelm one server

Try first: bigger instance, read replicas, caching, partitioning. Shard when those are exhausted.

STEP 2/6

Choosing a Shard Key

The most permanent decision in the design.

Good: tenant_id

high cardinality,■even spread,■queries filter by it

Bad: country

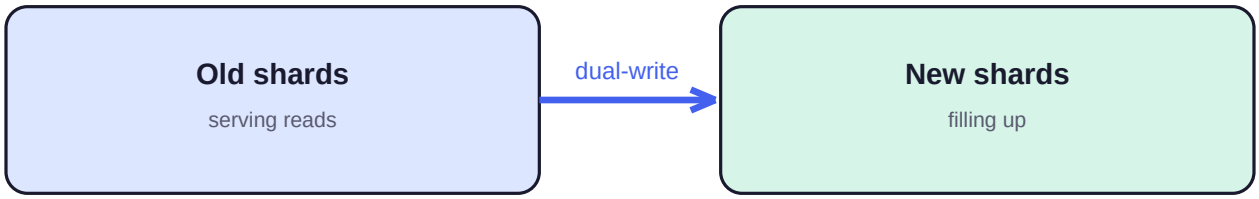
3 values ->■3 hot shards

- Cardinality: thousands+ distinct values so shards fill evenly.
- Locality: queries should hit ONE shard (avoid scatter-gather fan-out).
- Stability: keys that never change -- re-keying means moving data.

STEP 3/6

Resharding Strategies

Moving data while the system stays up.



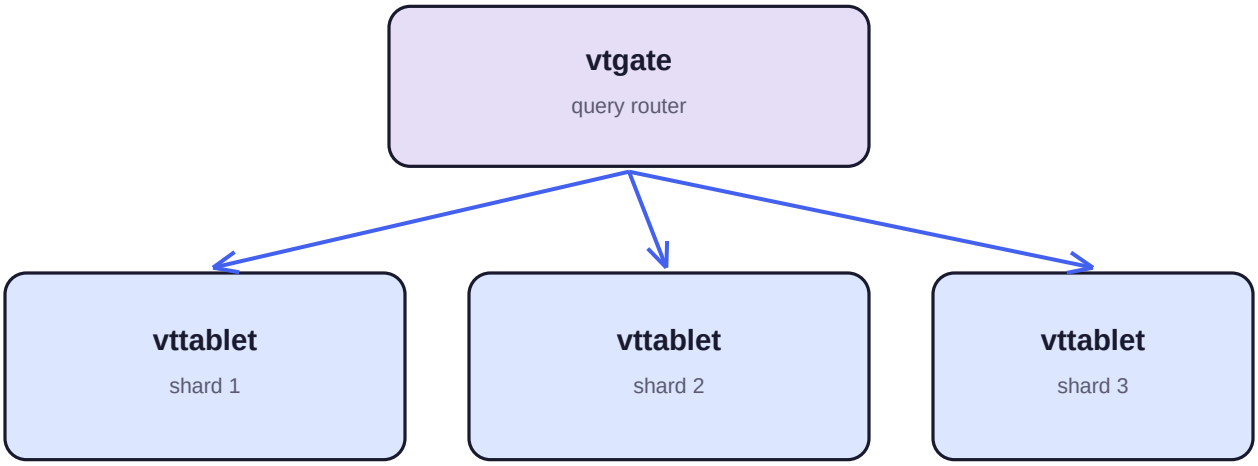
- Dual-write: new writes go to both; backfill copies history in batches.
- Verify counts/checksums per shard, then flip reads, then retire old shards.
- Range splits (split one hot shard) beat full rehashes -- less data moves.

Resharding is a migration project, not a config change -- plan verification like a deploy.

STEP 4/6

Vitess Architecture

Sharded MySQL behind a smart proxy.

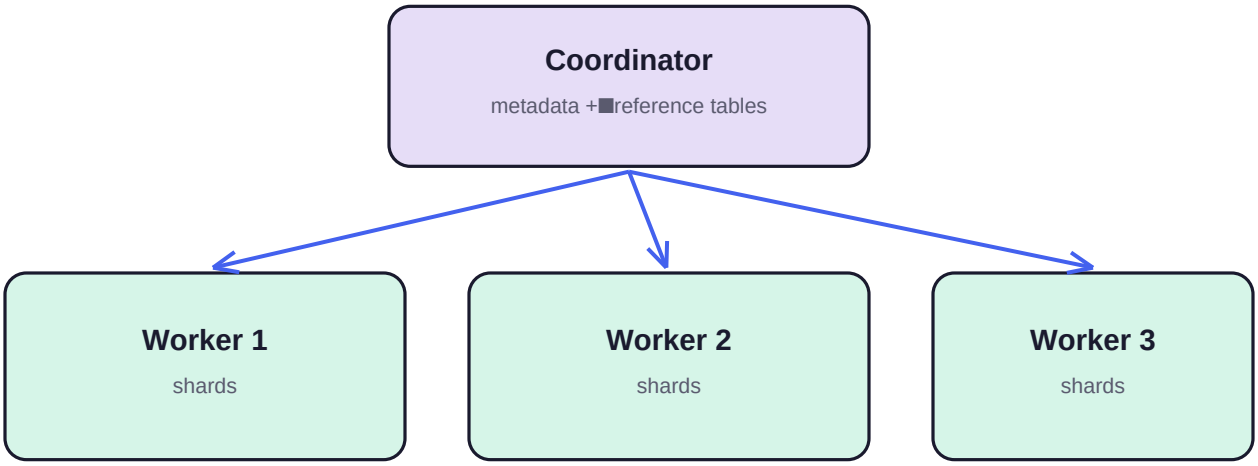


Apps speak MySQL protocol to vtgate; it routes, fans out, and reassembles. Battle-tested at YouTube scale.

STEP 5/6

Citus Architecture

Sharded Postgres with a coordinator.



- CREATE TABLE ... DISTRIBUTED BY (tenant_id) -- Postgres extension, not a fork.
- Co-located joins on the distribution key run fully on workers -- fast.
- Reference tables (small, replicated everywhere) avoid cross-shard joins.

STEP ✓I✓

Key Takeaways

Sharding in Practice -- Remember

- Shard last: exhaust bigger instances, replicas, caching, and partitioning first.
- Shard keys need cardinality, even spread, query locality, and stability -- choose once, carefully.
- Reshard with dual-writes + batched backfill + checksum verification, never a flag flip.
- Vitess shards MySQL behind a vtgate router; Citus distributes Postgres via coordinator + workers.
- Design queries to hit one shard -- scatter-gather across all shards kills the benefit.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev